

ALGORITHMS AND FLOWCHARTS

Why Algorithm is needed?

□ Computer Program ?

➤ *Set of instructions to perform some specific task.*

□ Is Program itself a Software ?

➤ *NO, Program is small part of software.*

□ Software merely comprises of Group of Programs (Source code)

□ **Code:** *refers to statements that are written in any programming language as machine code or C code.*

Why Algorithm is needed?

- Programming is both **tedious** and **exciting**.
- **Tedious** because like spoken languages *programming languages also have so many demanding rules.*
- **Exciting** because writing program provides the *programmer with the chance to create something new* also gives challenges of solving a problem.
- It is *quite difficult to write direct programs in any language*, just like you cannot start constructing building without the design of building.
- For constructing a building you need design of building, similarly for writing a large or good program you need algorithm.

METHODS OF SPECIFYING ALGORITHM

- **Pseudocode** - is a detailed yet *readable description of what a computer program or **algorithm** must do, expressed in a formally-styled natural language* rather than in a programming language.
- **Flowchart** - *a traditional graphical tool with standardized symbols*. Show the sequence of steps in an algorithm.

Program design



- **Program Design Process** has 2 phases:
 - ***Problem Solving Phase***
 - Creates an **algorithm** that solves the **problem**
 - ***Implementation (Coding) Phase***
 - **Translates** the algorithm into a **programming language**

Algorithms

- ☐ An *algorithm is a finite set of steps defining the solution of a particular problem.*
- ☐ *Need not to belong one particular language.*
- ☐ Sequence of English statements can also be algorithm.
- ☐ It is not a computer program
- ☐ *An algorithm can be expressed in English like language, called **pseudocode**, in a programming language or in the form of **flowchart**.*

PROPERTIES OF ALGORITHM

- **Finiteness** - there is an *exact number of steps to be taken and has an end.*
- **Absence of Ambiguity** - means that *every instruction is precisely described and clearly specified.*
- **Sequence of Execution** - instructions are performed *from top to bottom.*
- **Input and Output** - *defined the unknowns of the problem* is specified and *with the expected outcome.*
- **Effectiveness** - the *solution prescribed is guaranteed to give a correct answer* and that the specified process is faithfully carried out.
- **Scope Definition** - applies to a *specific problem or class of problem.*

Algorithm Vs Program

□ What is the difference between an algorithm and a program?

◇ *A program is an implementation of an algorithm* to be run on a specific computer and operating system.

◇ *An algorithm is more abstract* – it does not deal with machine specific details – *think of it as a method to solve a problem.*

- **What is good algorithm?**

Efficient algorithms are good, *we generally measure efficiency of an algorithm on the basis of:*

1. **Time:** algorithm should take minimum time to execute.
2. **Space:** algorithm should use less memory.

Algorithm Specifications

Every algorithm must satisfy the following criteria:

- **Input.** Zero or more *quantities are externally supplied*.
- **Output.** *At least one quantity is produced*.
- **Definiteness.** *Each instruction must be clear and unambiguous* (Unique meaning).
- **Finiteness.** An *algorithm terminates in a finite number of steps*.
- **Effectiveness.** *Every instruction must be* basic enough to be carried out than, means *not so complex*.

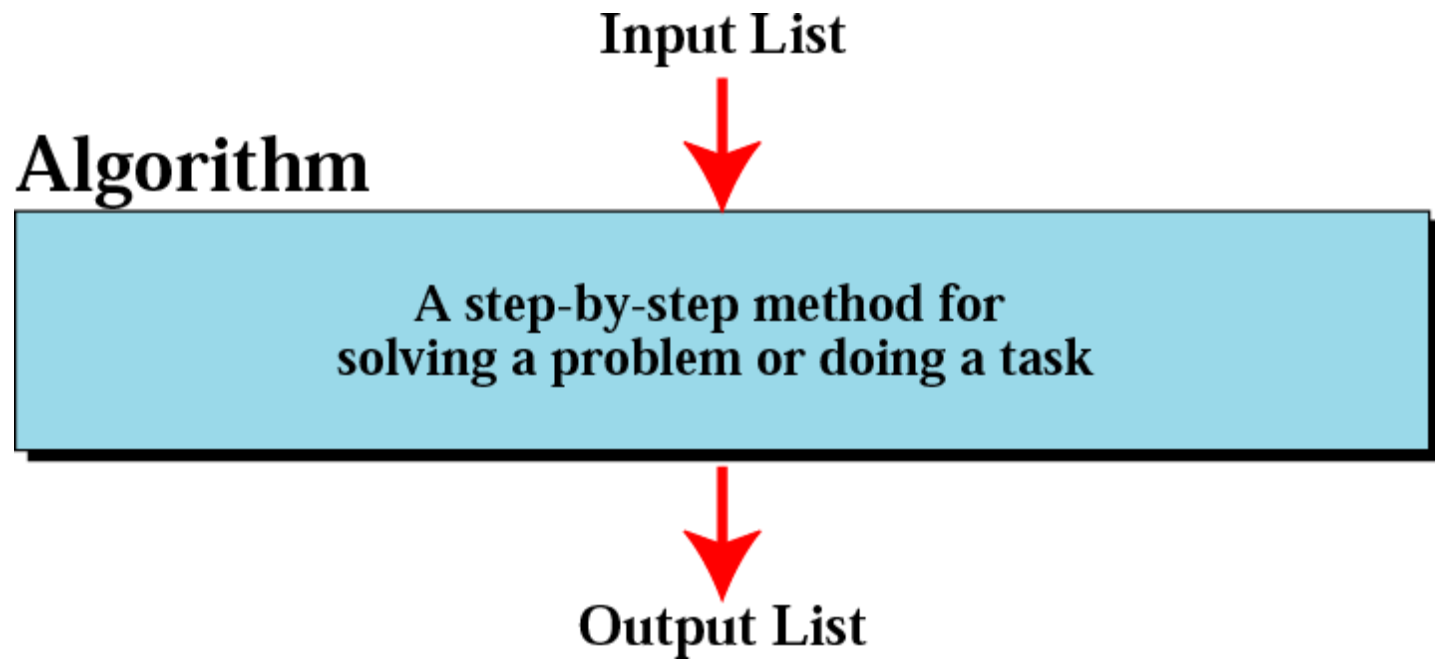
Pseudo code

- Pseudocode is *one of the methods that could be used to represent an algorithm*.
- It is *not written in a specific syntax that is used by a programming language and therefore cannot be executed in a computer*.
- There are lots of formats used for *writing pseudocodes* and most of them *borrow some of the structures from popular high level programming languages such as C, Lisp, FORTRAN*, etc.
- Also, *natural language is used in Pseudocode when presenting details that are not important*.
- Most of the algorithms are presented using pseudocode since they can be read and understood using programmers who are familiar with different programming languages.
- *Some programming constructs used for Pseudo Code – READ, PRINT, SET, INITIALISE, INCREMENT, IF – THEN – ENDIF, IF – THEN – ELSE – ENDIF, REPEAT – UNTIL, DO – WHILE etc.*

What is the difference between Algorithm and Pseudocode?

- *An algorithm is a well defined sequence of steps that provides a solution for a given problem, while a pseudocode is one of the methods that can be used to represent an algorithm.*
- *While algorithms can be written in natural language, pseudocode is written in a format that is closely related to high level programming language structures.*
- But pseudocode does not use specific programming language syntax and therefore could be understood by programmers who are familiar with different programming languages. Additionally, *transforming an algorithm presented in pseudocode to programming code could be much easier than converting an algorithm written in natural language to programming code.*

Informal definition of an algorithm



Finding the largest integer among five integers

12 8 13 9 11

Input List

FindLargest

Largest 12 12 List

Step 1

Largest 12 8 List

Step 2

Largest 13 13 List

Step 3

Largest 13 9 List

Step 4

Largest 13 11 List

Step 5

13

Output Result

Defining actions in Find Largest algorithm

12 8 13 9 11 Input List

FindLargest

Set Largest to the first number.

Step 1

If the second number is greater than Largest, set Largest to the second number.

Step 2

If the third number is greater than Largest, set Largest to the third number.

Step 3

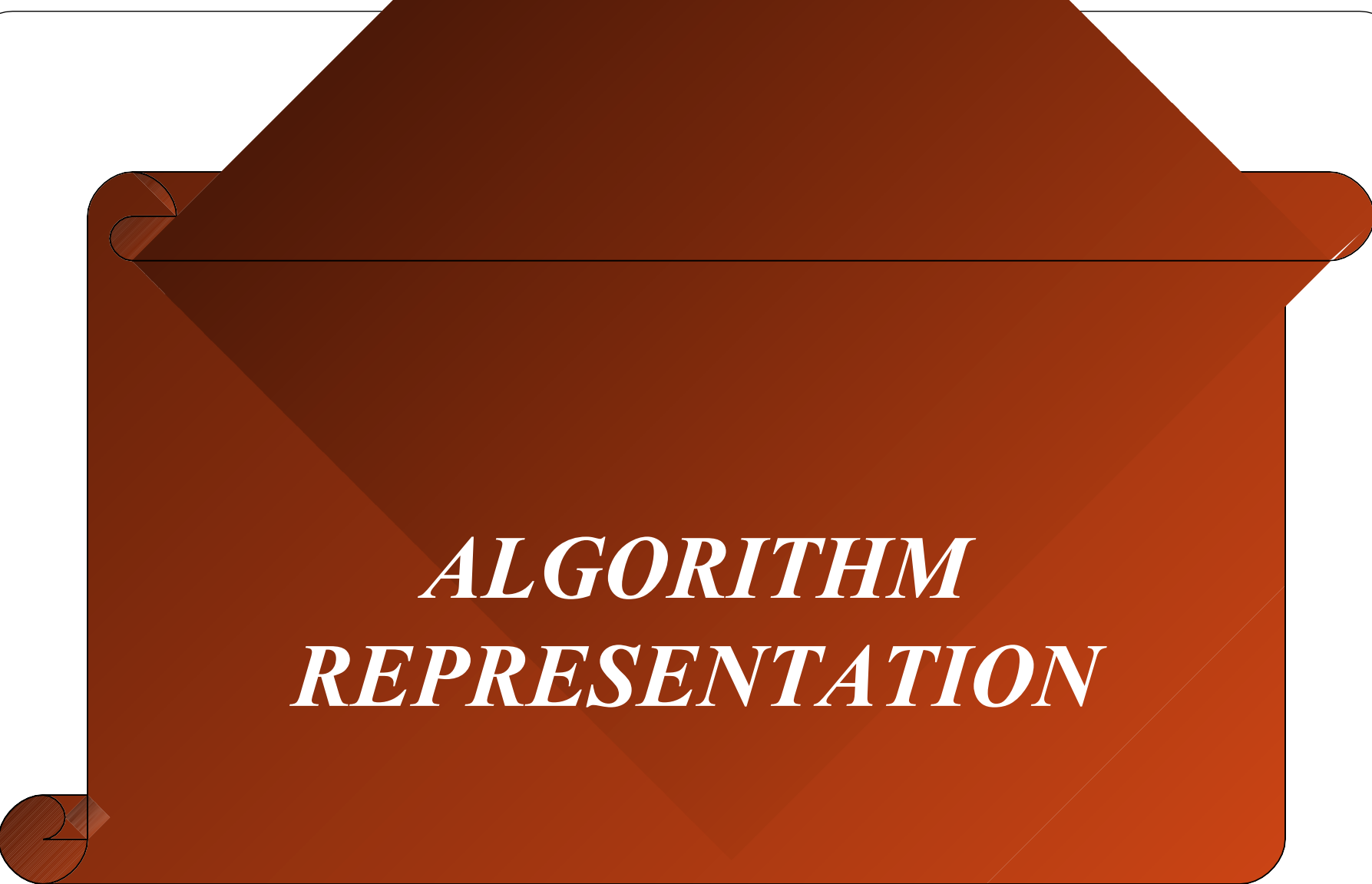
If the fourth number is greater than Largest, set Largest to the fourth number.

Step 4

If the fifth number is greater than Largest, set Largest to the fifth number.

Step 5

13 Output Result



ALGORITHM REPRESENTATION

Example 1

AverageOfTwo (Algorithm)

Input: Two numbers

- 1. Add the two numbers**
- 2. Divide the result by 2**
- 3. Return the result by step 2 (Output)**

End

Example 2

Pass/FailGrade

Input: One number

1. if (the number is greater than or equal to 40) then
 1. Set the grade to “pass”else
 2. Set the grade to “fail”End if
 2. Return the grade(**Output**)
- End**

Example 3

Marks range	Grade
≥ 80	A
$\geq 70 \ \& \ < 80$	B
$\geq 60 \ \& \ < 70$	C
$\geq 50 \ \& \ < 60$	D
< 50	F

Solution

Algorithm for Grading

AlgoForGrade

Input: One number

- 1. if (the number is between 80 and 100, inclusive)
then**
 - 1. Set the grade to “A”****End if**
- 2. if (the number is between 70 and 79, inclusive)
then**
 - 1. Set the grade to “B”****End if**

Continues on the next slide

3. if (the number is between 60 and 69, inclusive)
then

3.1 Set the grade to “C”

End if

4. if (the number is between 50 and 59, inclusive)
then

1. Set the grade to “D”

End if

5. If (the number is less than 50)
then

1. Set the grade to “F”

End if

6. Return the grade

End

Example 4

Solution

FindLargest

Input: 1000 positive integers

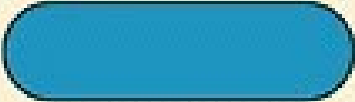
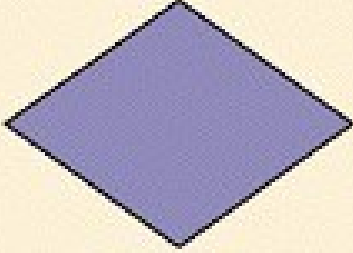
1. Set Largest to 0
2. Set Counter to 0
3. while (Counter less than 1000)
 1. if (the integer is greater than Largest) then
 1. Set Largest to the value of the integer End if
 2. Increment Counter
- End while
4. Return Largest

End

Flowchart

A graphical representation of an algorithm, often used in the design phase of programming to work out the *logical flow of a program*.

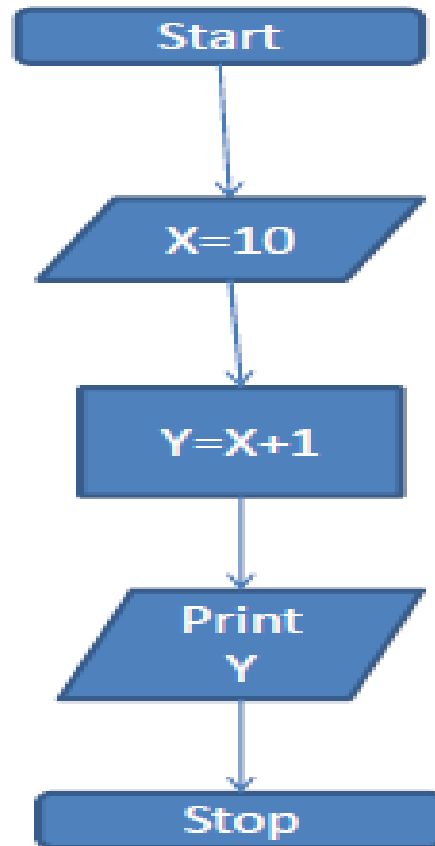
- ☐ Visual way to represent the information flow
- ☐ Make our logic more clear
- ☐ Help during writing of program
- ☐ Make testing and debugging easy

Name	Symbol	Use in flowchart
Oval		Denotes the beginning or end of a program.
Flow line		Denotes the direction of logic flow in a program.
Parallelogram		Denotes either an input operation (e.g., INPUT) or an output operation (e.g, PRINT).
Rectangle		Denotes a process to be carried out (e.g., an addition).
Diamond		Denotes a decision (or branch) to be made. The program should continue along one of two routes (e.g., IF/THEN/ELSE).

Flowchart Constructs

Sequence

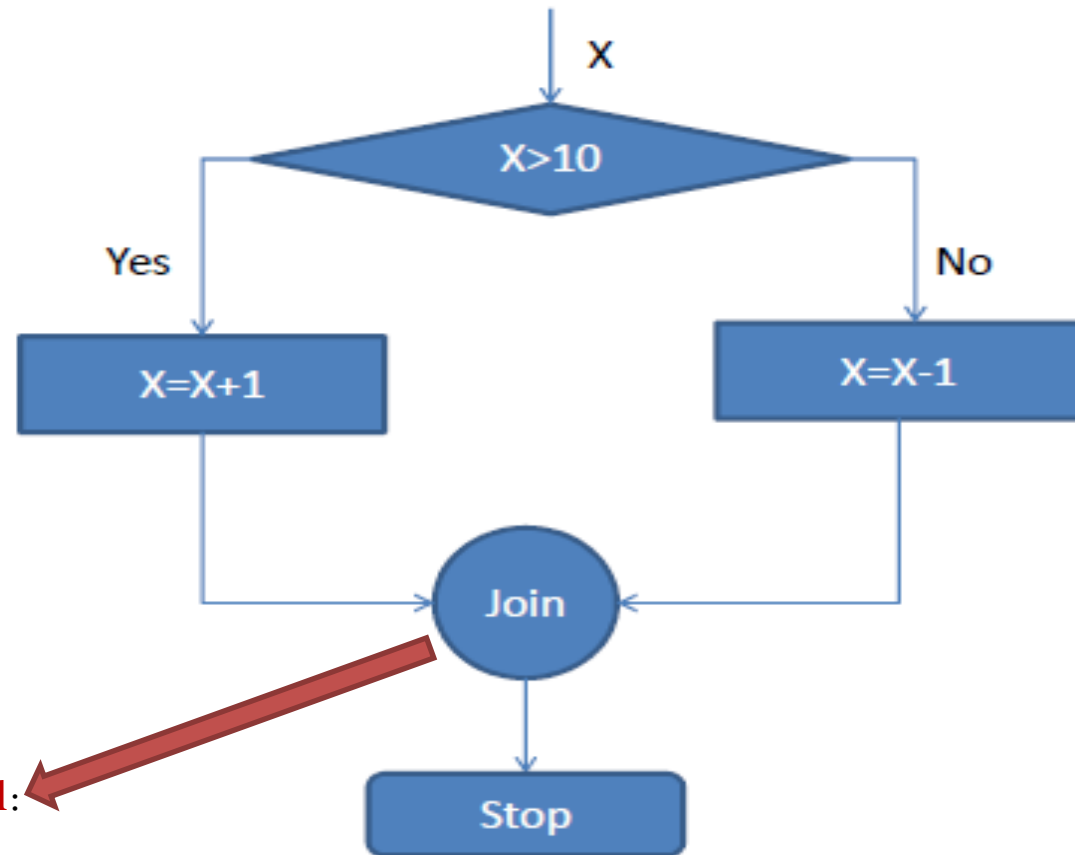
Sequence: In the sequence structure, statements are placed one after the other and the execution takes place starting from up to down.



Flowchart Constructs (cont..)

Selection

Selection (Branching): : In branch control, there is a condition and according to a condition, a decision of either TRUE or FALSE is achieved. In the case of TRUE, one of the two branches is explored; but in the case of FALSE condition, the other alternative is taken. Generally, the 'IF-THEN' is used to represent branch control.



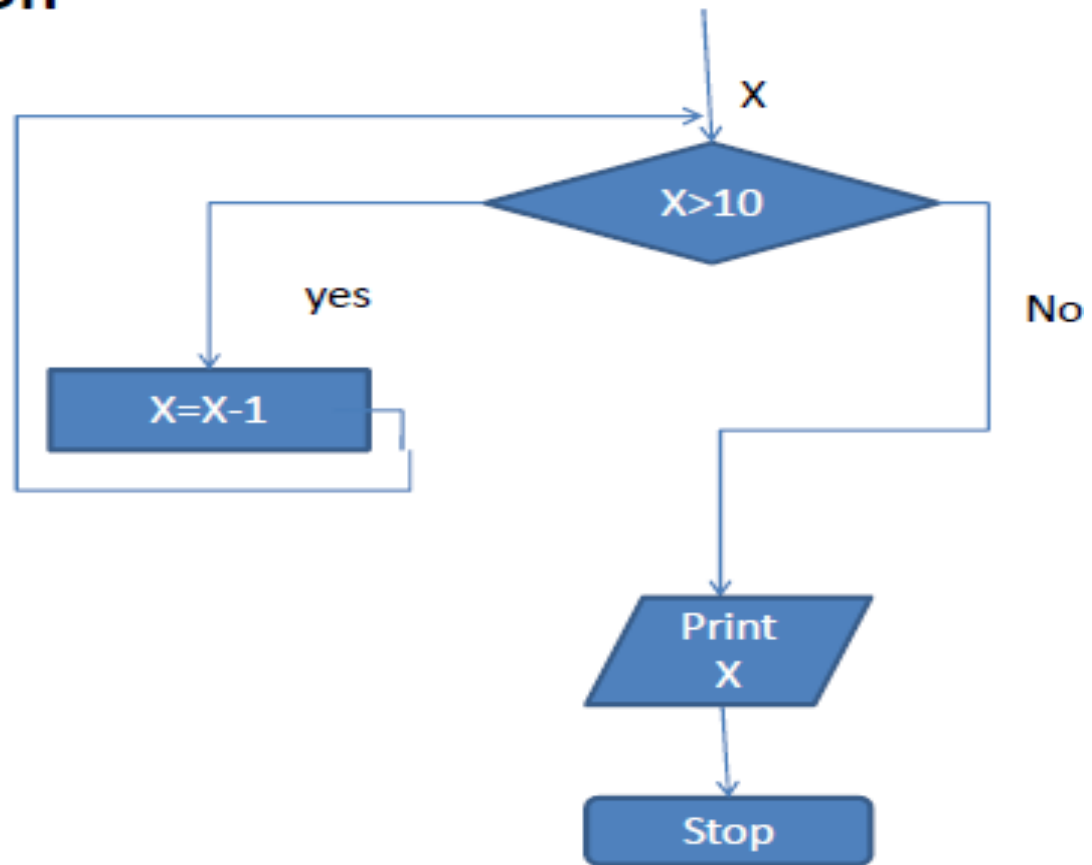
Connector Symbol:

When drawing only a portion of an algorithm

Flowchart Constructs (cont..)

Repetition

Repetition: The Loop or Repetition allows a statement(s) to be executed repeatedly based on certain loop condition e.g. WHILE, FOR loops.



Example-1

Write an algorithm and draw a flowchart that will read the two sides of a rectangle and calculate its area.

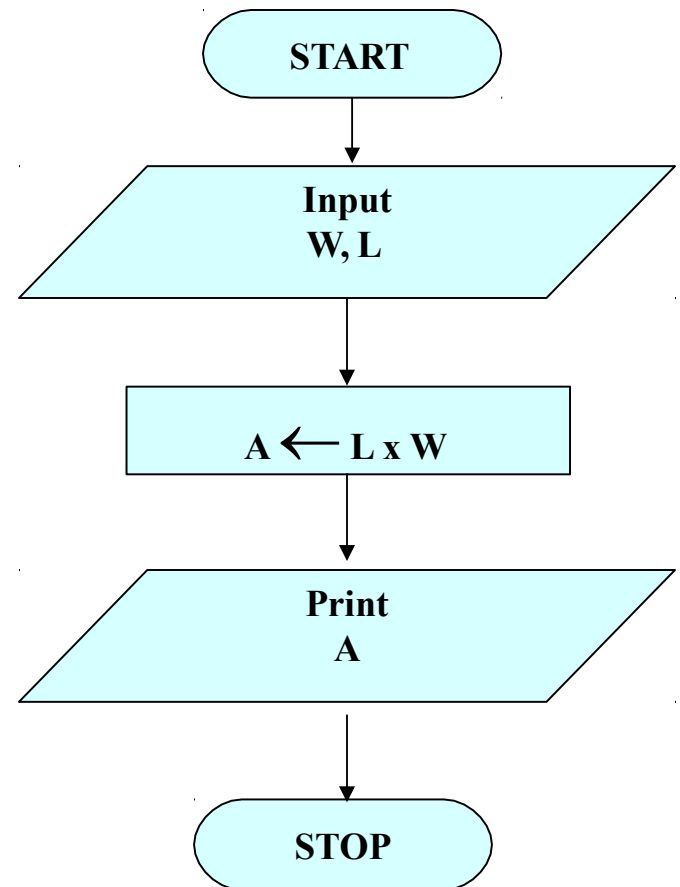
Algorithm

- Step 1: Take Width and Length as input
- Step 2 *Calculate Area by $Width * Length$*
- Step 3 Print Area.

Example-1

Pseudocode

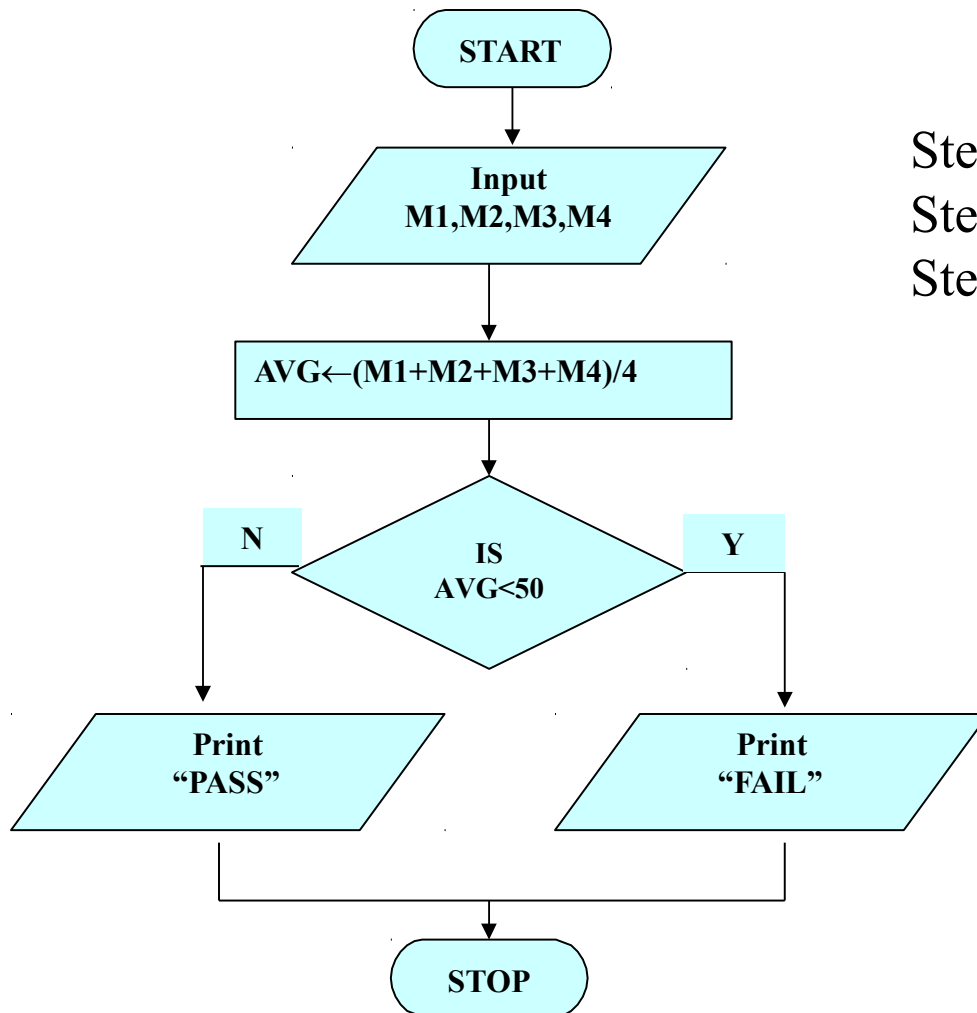
- Step 1: Input W,L
- Step 2: $A \leftarrow L \times W$
- Step 3: Print A



Example-2

- Write an Pseudocode and draw a flowchart that will take marks of four subjects and calculate the average. Then if average marks are greater than 50 then print PASS otherwise print FAIL.

Example-2



Step 1: Input M1,M2,M3,M4

Step 2: $AVG \leftarrow (M1 + M2 + M3 + M4) / 4$

Step 3: if (AVG < 50) then
Print "FAIL"

else

Print "PASS"

endif

Example 3

- Write an algorithm and draw a flowchart to convert the length in feet to centimeter.

Algorithm:

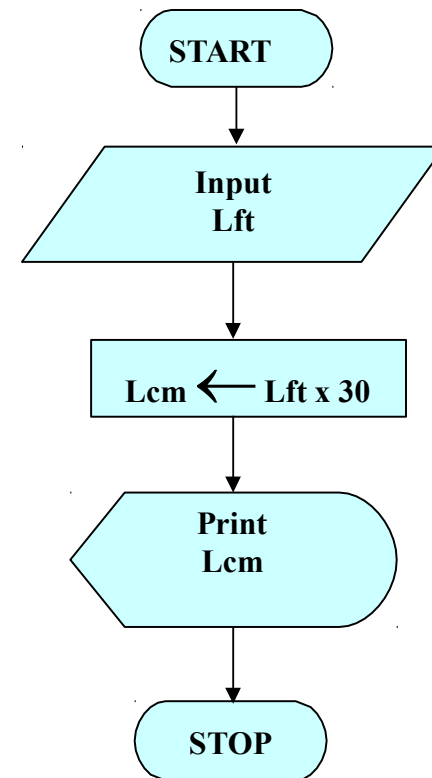
- *Input the length in feet (Lft)*
- *Calculate the length in cm (Lcm) by multiplying LFT with 30*
- *Print length in cm (LCM)*

Example 3

Pseudocode

- Step 1: Input Lft
- Step 2: $Lcm \leftarrow Lft \times 30$
- Step 3: Print Lcm

Flowchart



Example 4

- Write an algorithm and draw a flowchart that will calculate the roots of a quadratic equation

$$ax^2 + bx + c = 0$$

- Hint: $d = \text{sqrt}(b^2 - 4ac)$, and the roots are: $X_1 = (-b + d)/2a$ and $X_2 = (-b - d)/2a$

Example 4

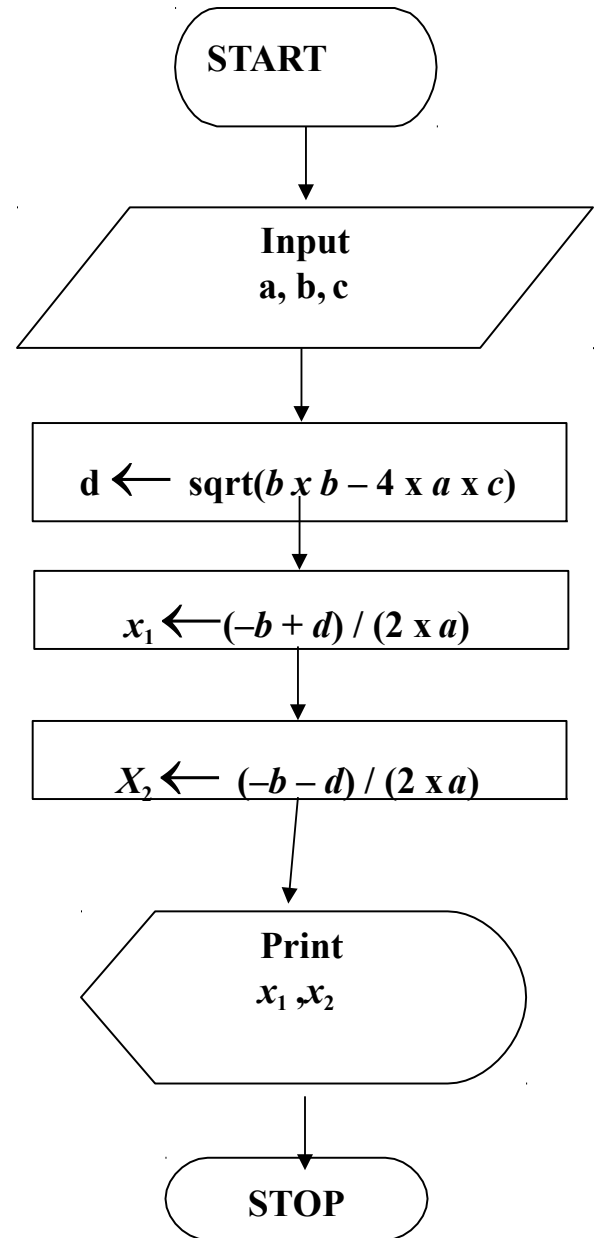
□ **Algorithm:**

- *Input the coefficients (a, b, c) of the quadratic equation*
- *Calculate $\mathbf{d}$*
- *Calculate $\mathbf{x}_1$*
- *Calculate $\mathbf{x}_2$*
- *Print $\mathbf{x}_1$ and $\mathbf{x}_2$*

Example 4

□ Pseudocode:

- Step 1: Input a, b, c
- Step 2: $d \leftarrow \text{sqrt}(b \times b - 4 \times a \times c)$
- Step 3: $x_1 \leftarrow (-b + d) / (2 \times a)$
- Step 4: $x_2 \leftarrow (-b - d) / (2 \times a)$
- Step 5: Print x_1, x_2



Example 5

- Write an algorithm that reads **three** numbers and prints the value of the largest number.

Example 5

Step 1: *Input* $N1, N2, N3$

Step 2: *if* ($N1 > N2$) *then*

if ($N1 > N3$) *then*

$MAX \leftarrow N1$ $// [N1 > N2, N1 > N3]$

else

$MAX \leftarrow N3$ $// [N3 > N1 > N2]$

endif

else

if ($N2 > N3$) *then*

$MAX \leftarrow N2$ $// [N2 > N1, N2 > N3]$

else

$MAX \leftarrow N3$ $// [N3 > N2 > N1]$

endif

endif

Step 3: *Print* “The largest number is”, MAX

Flow Charts Limitation

- ♣ For very large program, flow chart goes for many pages
- ♣ Costly to draw flow charts for large program
- ♣ Difficult to modify

EXERCISES

Sample 1: Write a program that calculates the sum of two input numbers and display the result.

Sample 2: Write a program to calculate the area of a circle and display the result. Use the formula: $A = \pi r^2$ where π is approximately equal to 3.1416.

Sample 3: Write a program that computes the average of three input quizzes, and then display the result.

Sample 4: Write a program that converts the input Fahrenheit degree into its Celsius degree equivalent. Use the formula: $C = (5/9) * F - 32$.

Sample 5: Create a program to compute the volume of a sphere. Use the formula: $V = (4/3) * \pi r^3$ where π is equal to 3.1416 approximately. The r^3 is the radius. Display result.

Sample 6: Write a program that converts the input Celsius degree into its equivalent Fahrenheit degree. Use the formula: $F = (9/5) * C + 32$.